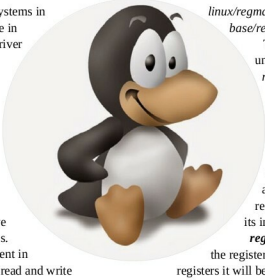
regmap: Reducing the Redundancy in Linux Code

This article gives an overview of regmap—how useful it is in factoring out common code from the Linux sub-system and how to effectively use it to write drivers in Linux.

Linux is divided into many sub-systems in order to factor out common code in different parts and to simplify driver development, which helps in code maintenance.

Linux has sub-systems such as I2C and SPI, which are used to connect to devices that reside on these buses. Both these buses have the common function of reading and writing registers from the devices connected to them. So the code to read and write these registers, cache the value of the registers, etc, will have to be present in both these sub-systems. This causes redundant code to be present in all sub-systems that have this register read and write functionality.



To avoid this and to factor out common code, as well as for easy driver maintenance and development, Linux developers introduced a new kernel API from version 3.1, which is called regmap. This infrastructure was previously present in the Linux AsoC (ALSA) sub-system, but has now been made available to entire Linux through the regmap API.

Earlier, if a driver was written for a device residing on the SPI bus, then the driver directly used the SPI bus read and write calls from the SPI sub-system to talk to the device. Now, it uses the regmap API to do so. The regmap sub-system takes care of calling the relevant calls of the SPI sub-system. So two devices—one residing on the I2C bus and one on the SPI bus—will have the same regmap read and write calls to talk to their respective devices on the bus.

This sub-system was first introduced in Linux 3.1 and later, the following different features were introduced:

- Support for SPMI, MMIO
- Spinlock and custom lock mechanism
- Cache support
- Endianness conversion
- Register range check
- IRQ support
- Read only and write only register
- Precious register and volatile register
- Register pages

Implementing regmap

regmap in Linux provides APIs that are declared in *include/linux/regmap.h* and implemented in *drivers/base/regmap/*.

Two important data structures to understand in Linux regmap are struct *regmap_config* and struct *regmap*.

struct *regmap_config*

This is a per device configuration structure used by the regmap sub-system to talk to the device. It is defined by driver code, and contains all the information related to the registers of the device. Descriptions of its important fields are listed below.

reg_bits: This is the number of bits in the registers of the device, e.g., in case of 1 byte registers it will be set to the value 8.

val_bits: This is the number of bits in the value that will be set in the device register.

writeable_reg: This is a user-defined function written in driver code which is called whenever a register is to be written. Whenever the driver calls the regmap sub-system to write to a register, this driver function is called; it will return 'false' if this register is not writeable and the write operation will return an error to the driver. This is a 'per register' write operation callback and is optional.

wr_table: If the driver does not provide the *writeable_reg* callback, then *wr_table* is checked by regmap before doing the write operation. If the register address lies in the range provided by the *wr_table*, then the write operation is performed. This is also optional, and the driver can omit its definition and can set it to NULL.

readable_reg: This is a user defined function written in driver code, which is called whenever a register is to be read. Whenever the driver calls the regmap sub-system to read a register, this driver function is called to ensure the register is readable. The driver function will return 'false' if this register is not readable and the read operation will return an error to the driver. This is a 'per register' read operation callback and is optional.

rd_table: If a driver does not provide a *readable_reg* callback, then the *rd_table* is checked by regmap before doing the read operation. If the register address lies in the range provided by *rd_table*, then the read operation is performed. This is also optional, and the driver can omit its definition and can set it to NULL